

A — Frog 1

State: $dp[i]$ = minimum cost to reach stone i .

Only jumps of length 1 or 2 are allowed, so transition from $i-1$ and $i-2$ with the corresponding height difference. **Trick:** Basic minimum-cost-on-a-line DP.

Complexity: $O(N)$.

B — Frog 2

State: $dp[i]$ = minimum cost to reach stone i .

Use $dp[i] = \min_{1 \leq j \leq K} \{dp[i-j] + |h_i - h_{i-j}|\}$ over valid j . **Trick:** If each position depends on the previous K states, plain $O(NK)$ DP is often enough.

Complexity: $O(NK)$.

C — Vacation

State: $dp[i][a]$ = best happiness through day i if activity a is chosen today. Transition only from the other two activities, since consecutive equal choices are forbidden. **Trick:** One option per day + forbidden consecutive repeats. **Complexity:** $O(N)$.

D — Knapsack 1

State: $dp[w]$ = maximum value achievable with weight at most w .

For each item, iterate w **backwards**; otherwise the current item could be reused. **Trick:** Canonical 0/1 knapsack when capacity W is small.

Complexity: $O(NW)$.

E — Knapsack 2

Weight is huge but total value is small. **State:** $dp[v]$ = minimum weight needed to obtain value v .

Answer the largest v with $dp[v] \leq W$. **Trick:** Invert the knapsack dimension when one bound is too large.

Complexity: $O(N \sum v_i)$.

F — LCS

State: $dp[i][j]$ = LCS length of prefixes $s[0..i)$ and $t[0..j)$.

If characters match, go diagonally +1; otherwise drop one character and take the better state. **Trick:** Reconstruct by walking backward through the DP table.

Complexity: $O(|S||T|)$.

G — Longest Path

Since the graph is a DAG, process vertices in topological order. **State:** $dp[v]$ = longest path ending at v .

Relax each edge $v \rightarrow u$ with $dp[u] = \max(dp[u], dp[v] + 1)$. **Trick:** DAG longest path is just DP after topological ordering.

Complexity: $O(N + M)$.

H — Grid 1

State: $dp[i][j]$ = number of paths to cell (i, j) .

For an open cell, add from top and left; for a wall, set the state to zero. **Trick:** Only previous-row/current-left values matter, so compress to $O(W)$ memory.

Complexity: $O(HW)$.

I — Coins

State: $dp[j]$ = probability of exactly j heads after the processed coins.

Each coin splits every state into heads and tails transitions; finally sum states with $j > n - j$. **Trick:** Probability DP on number of successes.

Complexity: $O(N^2)$.

J — Sushi

Only counts (a, b, c) of plates with 1, 2, and 3 sushi matter; plate identities do not. Empty-plate picks cause a self-loop, yielding $E = (N + aE_1 + bE_2 + cE_3)/(a + b + c)$. **Trick:** Compress symmetric states, then algebraically remove the expectation self-loop.

Complexity: $O(N^3)$ states.

K — Stones

State: $win[x]$ = whether the player to move with x stones can force a win.

A state is winning iff **some** legal move reaches a losing state. **Trick:** Standard impartial-game win/lose DP.

Complexity: $O(NK)$.

L — Deque

Store the optimal **score difference**, not both players' totals. For interval $[l, r]$: $dp[l][r] = \max(a_l - dp[l+1][r], a_r - dp[l][r-1])$. **Trick:** Score-difference DP turns alternating optimal play into a one-player recurrence.

Complexity: $O(N^2)$.

M — Candies

State: $dp[j]$ = ways to distribute exactly j candies after some children.

Each child's transition sums a contiguous range of previous states. **Trick:** Prefix sums make each range sum $O(1)$, reducing $O(NK^2)$ to $O(NK)$.

Complexity: $O(NK)$.

N — Slimes

State: $dp[l][r]$ = minimum cost to merge all slimes in $[l, r)$ into one.

Split at every m : $dp[l][r] = \min_m \{dp[l][m] + dp[m][r] + \text{sum}(l, r)\}$. **Trick:** Classic interval DP; prefix sums give interval merge cost in $O(1)$.

Complexity: $O(N^3)$.

O — Matching

Use a mask of already-used women; the next man is automatically popcount($mask$). Try every compatible unused woman and set her bit. **Trick:** Standard assignment DP for $N \lesssim 20$.

Complexity: $O(N2^N)$.

P — Independent Set

Tree DP with two states per vertex: white or black. A black vertex forces every child white; a white vertex allows either child color, and independent child contributions multiply. **Trick:** Two-color tree DP with local compatibility constraints.

Complexity: $O(N)$.

Q — Flowers

This is weighted LIS: for height h_i , query the best beauty ending at a **smaller** height, then add a_i . **Trick:** Fenwick/segment tree stores prefix maximums instead of LIS lengths.

Complexity: $O(N \log N)$.

R — Walk

For adjacency matrix A , $(A^K)_{ij}$ counts walks of exactly K edges from i to j .

Trick: “Count walks of huge exact length” $\Rightarrow$ binary matrix exponentiation; sum all entries of A^K .

Complexity: $O(N^3 \log K)$.

S — Digit Sum

Digit DP state is (position, digit-sum mod D , tight). Leading zeros represent shorter numbers automatically; subtract one at the end to remove zero. **Trick:** Standard upper-bound digit DP with a modular aggregate.

Complexity: $O(|K|D \cdot 10)$.

T — Permutation

Build relative order one element at a time; $dp[j]$ tracks the rank of the newest element among the current prefix. $A <$ transition is a prefix sum of ranks, while $>$ is a suffix sum. **Trick:** Cumulative sums reduce the naive $O(N^3)$ recurrence to $O(N^2)$.

Complexity: $O(N^2)$.

U — Grouping

Precompute $score[mask]$, the total pair contribution when exactly that subset forms one group. Then $dp[mask] = \max_{sub \subseteq mask} \{score[sub] + dp[mask \setminus sub]\}$. **Trick:** Force sub to contain one fixed bit of $mask$ to avoid equivalent partitions.

Complexity: $O(3^N)$.

V — Subtree

Compute contributions from children, then reroot so each node also receives the contribution from its parent side. When sending from v to child u , multiply all neighbor contributions except u . **Trick:** Prefix/suffix products exclude one child in $O(1)$ without modular division; modulus is arbitrary.

Complexity: $O(N)$.

W — Intervals

Process positions left-to-right; each state is the position of the most recent selected 1. An interval $[l, r]$ ending at r adds its value to exactly a contiguous range of states. **Trick:** Range-add + global-max DP $\Rightarrow$ lazy segment tree.

Complexity: $O((N + M) \log N)$.

X — Tower

Sort blocks by $w + s$. Let $dp[x]$ be the best value when total weight x is already above the current block; place it underneath only if $x \leq s$. **Trick:** The ordering argument turns the problem into ordinary 0/1 knapsack.

Complexity: $O(N \cdot \max(w + s))$.

Y — Grid 2

Obstacle-free monotone paths between two cells are binomial coefficients. Sort blocked cells plus the destination; for each point, start with all paths to it and subtract paths that pass through each earlier blocked point. **Trick:** Combinatorics + DP over only the special points.

Complexity: $O(N^2 + H + W)$.

Z — Frog 3

Expand $dp[i] = \min_j \{dp[j] + (h_i - h_j)^2 + C\}$ into $h_i^2 + C + \min_j \{(-2h_j)h_i + dp[j] + h_j^2\}$. Each previous j becomes a line queried at $x = h_i$. **Trick:** Increasing heights give monotone slopes and queries, so deque CHT gives $O(N)$ total.